

浙江大学



本科实验报告

课程名称:	C++项目管理与工程实践
实验名称:	验收总结报告
姓 名:	刘欣瓚 石耿同
专 业:	计算机科学与技术
指导教师:	袁昕
报告日期:	2026/7/18

Table of Contents

一、成果展示	3
一、 任务管理主界面与编辑器	3
二、 依赖图可视化	4
三、 数据统计仪表盘	6
四、 桌面宠物	6
五、 专注计时	7
六、 项目开发工具展示	8
6.1) git 版本控制	8
二、智能体使用说明	9
一、 智能体配置与使用方式	9
二、 MCP Server 设置	9
三、 规则文档编写	9
四、 技能文档编写	10
五、 每轮对话与开发记录	10
六、 团队协作情况	11
6.1) 分工安排	11
6.2) 人与智能体协同工作	11
七、 实际效果	12
八、 现存问题	12
三、学习回顾与个人总结	12
一、 刘欣瓚	12
二、 石耿同	13
四、课程改进建议	13

一、成果展示

SmartMate 是一款基于 C++20、Qt 6 Widgets 和 SQLite 的 Windows 桌面任务规划应用。它不仅记录待办，还通过任务状态、优先级、截止时间和前后依赖回答“现在应该先做什么”，并以主窗口、桌宠、专注和统计组成“规划—执行—记录—复盘”闭环。以下截图展示最终版本的实际运行效果。

一、任务管理主界面与编辑器

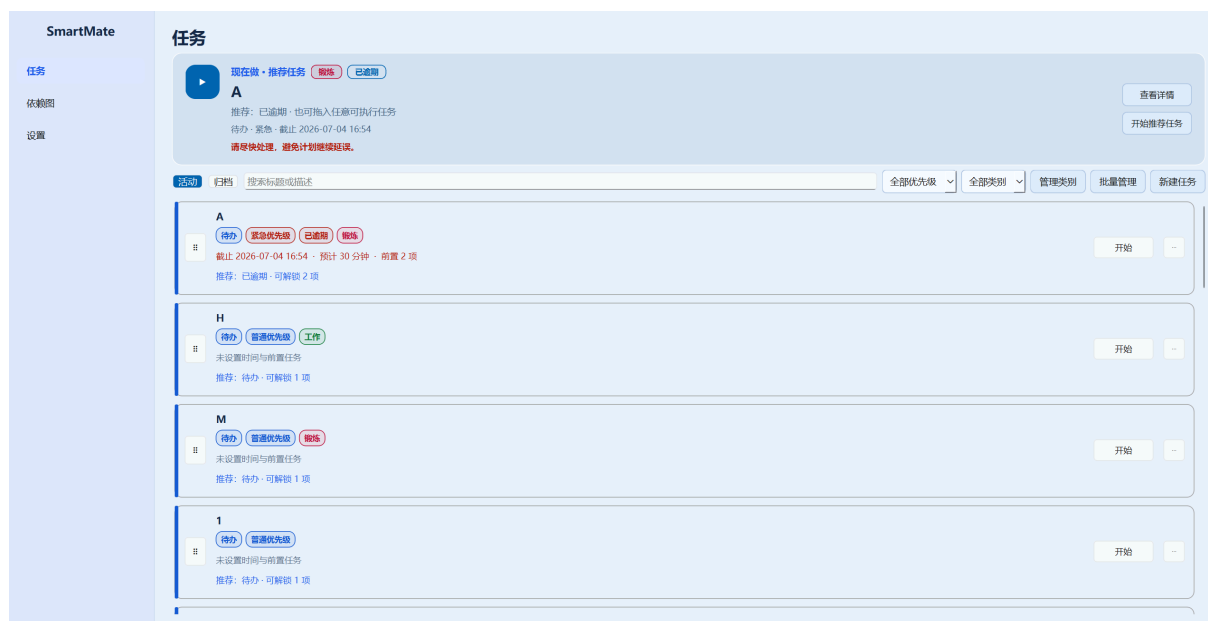


Figure 1: 任务管理主界面。顶部“现在做”聚焦槽展示系统推荐的最优先任务并支持拖拽开始；中部为组合筛选工具栏（关键字搜索 + 优先级 + 类别）；下方任务卡片由 TaskItemDelegate 自绘，徽标式标签清晰区分状态和属性。

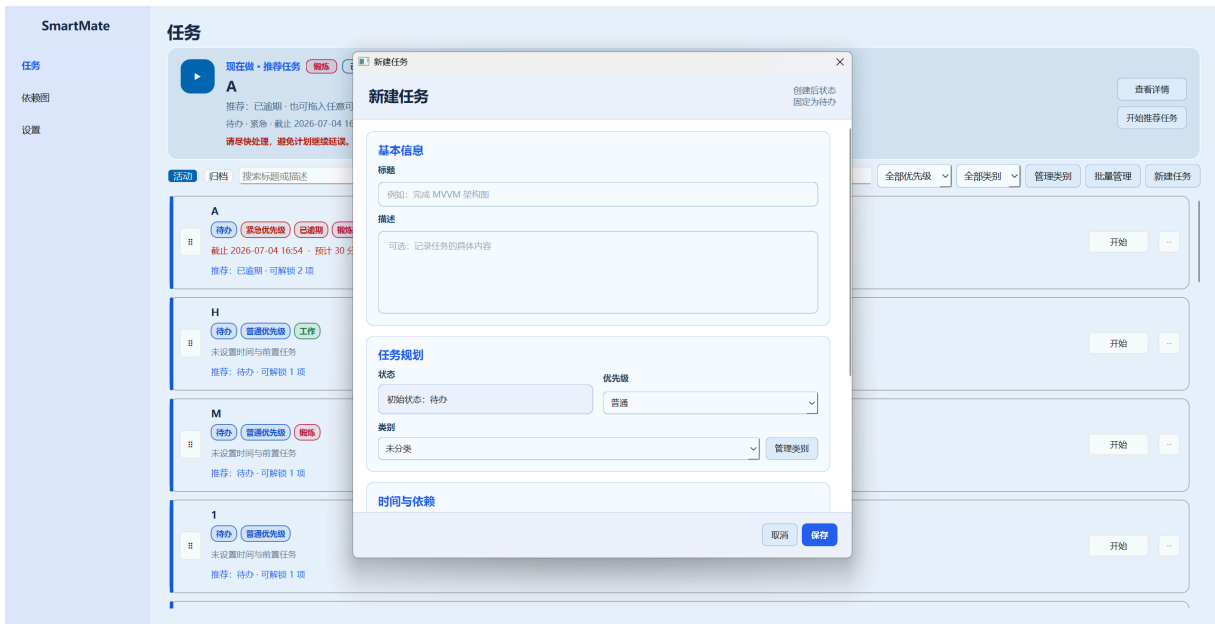


Figure 2: 任务编辑器。状态只读展示，新建任务固定为“待办”。表单包含标题、描述、优先级、类别（下拉框 + 跳转管理弹窗）、前置任务选择、截止时间选择器（日历 + 24 小时制时间）和预计用时选择器（天/小时/分钟三栏）。

二、依赖图可视化

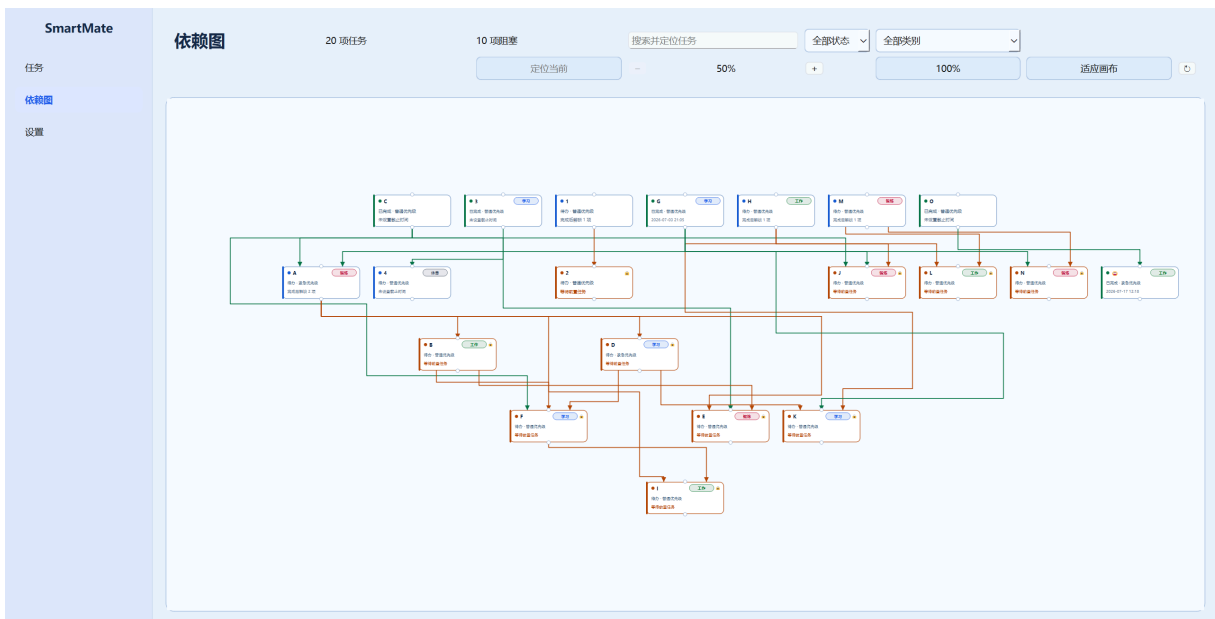


Figure 3: Finish-to-Start 依赖图，使用 QGraphicsView + QGraphicsObject 渲染。节点按 Kahn 拓扑层级从左到右排列，边使用正交折线路由，绿色 = 已满足（前置 Done），橙色 = 未满足（阻塞）。右侧详情面板展示选中节点的前置/后继数量和阻塞原因，支持从图页直接打开依赖编辑器。

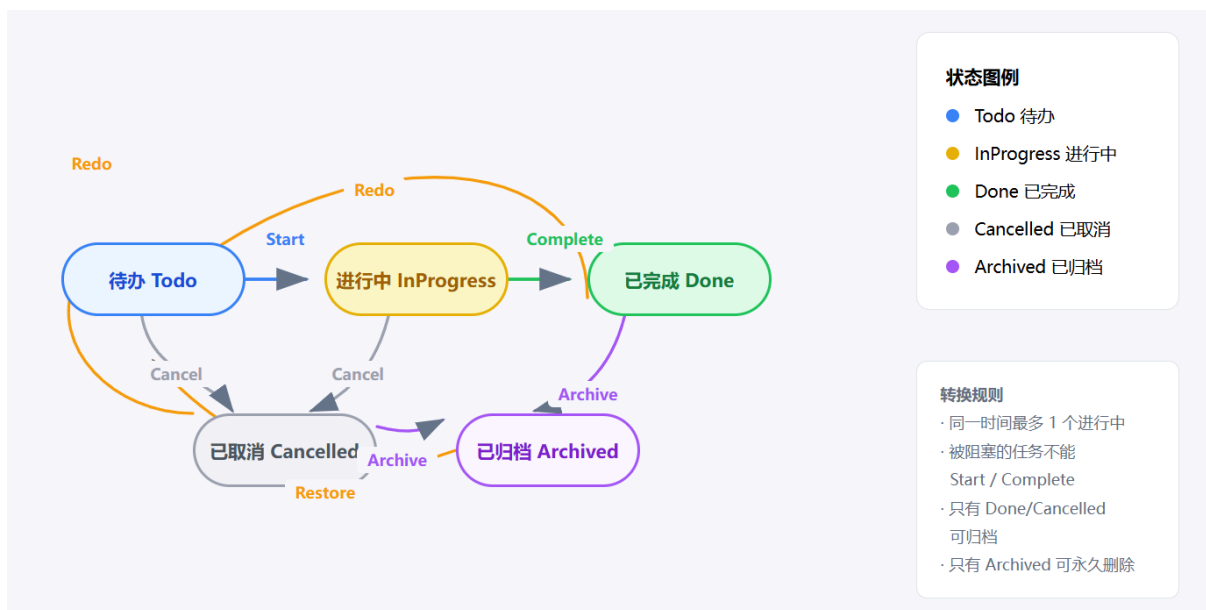


Figure 4: 任务状态机转换图。核心约束：同一时间最多一个进行中；被阻塞任务不能 Start/Complete；只有 Done/Cancelled 可归档；只有 Archived 可永久删除。

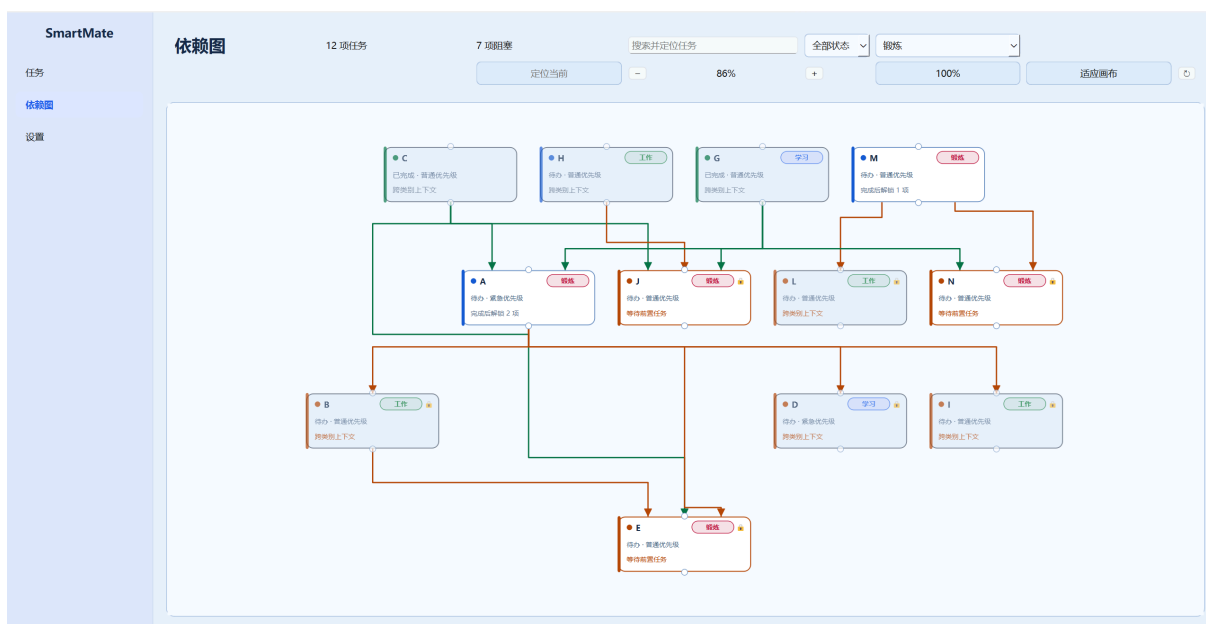
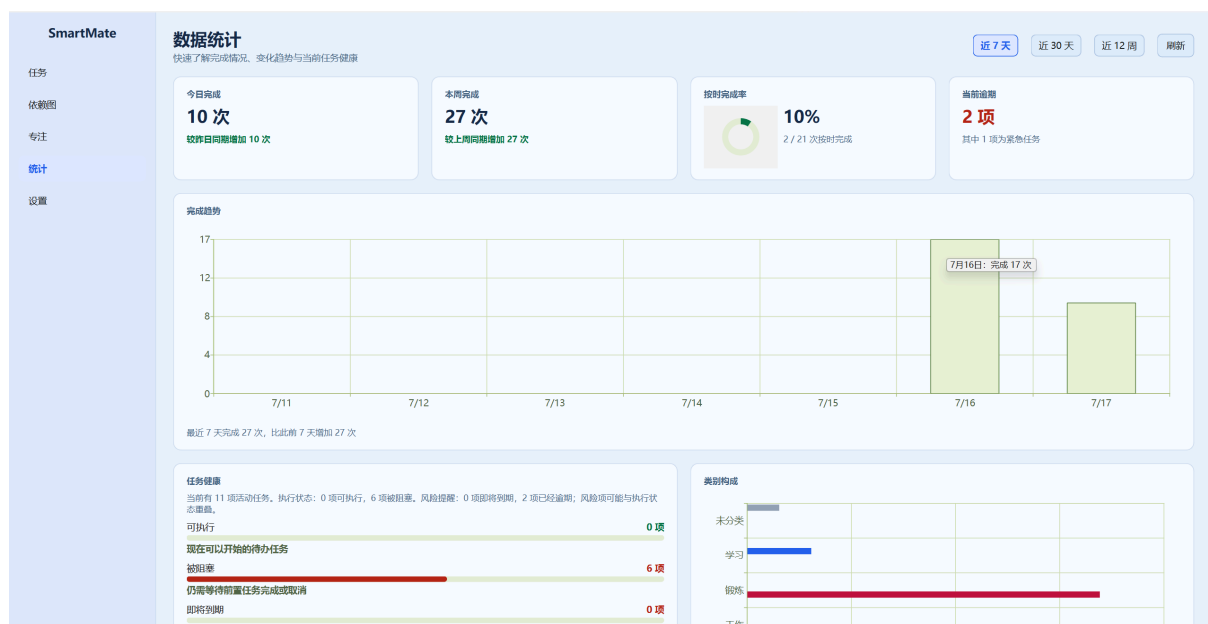


Figure 5: 依赖图中也可以用类别来显示。其中白色的为该类别的任务，浅色的为该类别任务绑定的相关依赖任务。“。

三、数据统计仪表盘



四、桌面宠物



Figure 7: 桌面宠物双形态。主窗口可见时, 趴姿三花猫锚定在窗口上边缘(鼠标穿透, 只做装饰)。



Figure 8: 最小化时切换为可拖动的坐姿浮动窗口，点击弹出任务气泡显示当前推荐任务，支持「开始」和「完成」快捷操作。位置通过 QSettings 持久化，支持多屏恢复。

五、专注计时



Figure 9: 专注计时页面。仅对当前 InProgress 任务生效，支持开始→暂停→继续→完成/放弃的完整生命周期。计时器每秒递增，暂停时段不计入总时长；完成后的会话进入历史列表，显示任务标题、累计时长、起止时间和类别。5 秒检查点机制保障异常退出后可恢复。

六、 项目开发工具展示

6.1) git 版本控制

Commits on Jul 16, 2026

Merge: add desktop pet Nemoun committed 20 hours ago	ed020f2		
fix: enhance desktop pet positioning and appearance imperfect-jade committed 20 hours ago	2e77178		
feat: add floating desktop pet feature with task popup and sprite animation imperfect-jade committed yesterday	89ffd48		
update functional-test Nemoun committed yesterday	03462ad		
Merge: add Task Activity Event and Statistics Service Nemoun committed yesterday	6f2b385		
fix: fix bugs of charts and texts in statistics module imperfect-jade committed yesterday	407b012		
feat: add StatisticsPage widget and corresponding tests imperfect-jade committed yesterday	41617f6		
feat: Implement Statistics Projection ViewModel imperfect-jade committed yesterday	c3b1f70		
feat: Implement Task Activity Event and Statistics Service imperfect-jade committed yesterday	c2feb98		
refactor: Enhance task details presentation imperfect-jade committed 2 days ago	3824aa3		

Figure 10: github 上相关的 commit 内容，展示每次迭代和相关的工作。

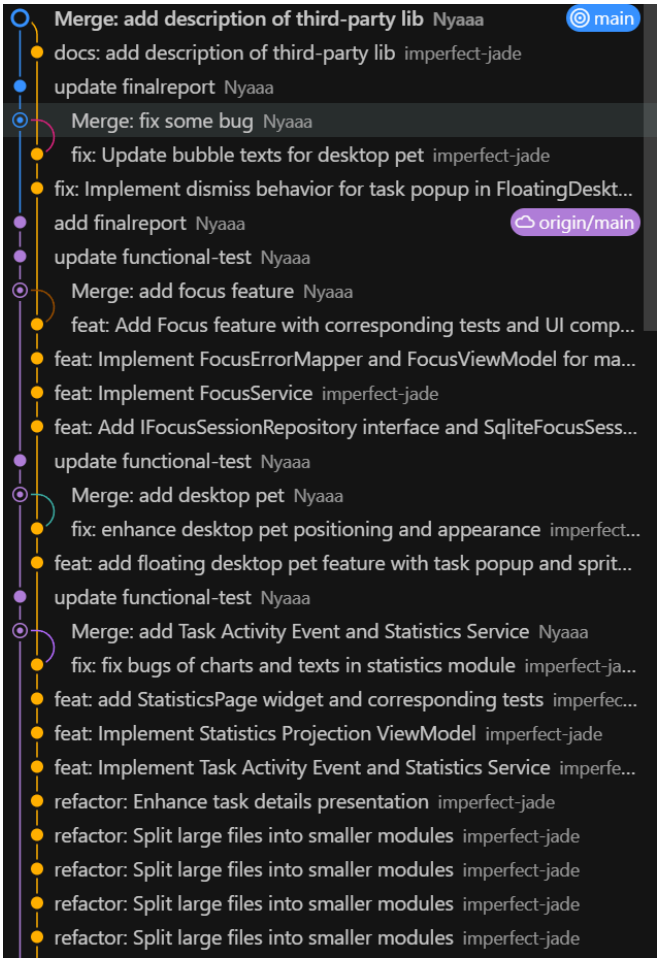


Figure 11: vscode 开发的源代码管理图，展示每次提交的内容。

二、智能体使用说明

一、智能体配置与使用方式

项目采用“开发者负责目标、边界和验收，智能体负责仓库分析、实现、测试和文档协作”的模式。Shell 为 PowerShell，工具链为 Qt 6.10.2 配套 MinGW 13.1 与 CMake preset。

每轮工作遵循以下流程：

1. 读取 `AGENTS.md` 和目标模块设计文档；
2. 检查 Git 工作区，保护开发者未提交修改；
3. 修改前说明所属层、目标数据流和测试方式；
4. 检索已有接口、Fake 和测试，不平行创造第二套机制；
5. 小步实现并同步公共注释、架构和设计文档；
6. 先运行相关测试，再执行完整 CTest，发布相关任务再执行部署脚本；
7. 汇报变更、证据、限制与遗留事项，由开发者最终验收。

智能体不能自行放宽架构约束，不能用 EventBus、Controller、Service Locator 或字符串反射路由换取短期速度，不能覆盖无关改动，也没有独立发布或产品决策权。

二、MCP Server 设置

仓库中没有 MCP Server 配置，也没有依赖某个远程 MCP 服务才能完成构建。当前开发的核心能力来自本地文件访问、PowerShell、CMake/CTest 和 Git，因此其他开发者克隆仓库后仍可独立复现项目。

三、规则文档编写

`AGENTS.md` 是本项目最重要的智能体规则文档。它将架构原则转化为可执行约束，主要包括：

- 固定编译期依赖与运行时数据流；
- Qt Widgets 为唯一正式前端，禁止重新引入 QML、Quick 和 .ui；
- Common、Model、Persistence、Contracts、ViewModel、Widget 和 App 的职责；
- 稳定 ID、强类型命令、通知信号、初始同步和双向绑定规则；
- 任务状态机、依赖、类别、批量事务和图布局的归属；
- 中文注释规范、修改前说明和修改后验收命令；
- 禁止 Controller、EventBus、Service Locator、全局业务单例和跨层捷径。

与简短提示词相比，规则文档的价值在于它长期存在于仓库中，可被每轮智能体会话重新读取，也能被人工评审和架构测试交叉验证。

四、技能文档编写

现阶段的技能文档主要分散在 AGENTS.md、docs/architecture.md、docs/widgets-migration.md 和 docs/windows-deployment.md 中。docs/architecture.md 解释架构原因；widgets-migration.md 记录 QML→Widgets 迁移；desktop-pet-design.md、statistics-design.md 和 focus-design.md 固化后期模块的数据事实、Contract 与验收边界；windows-deployment.md 记录工具链和发布排障。

五、每轮对话与开发记录

日期/轮次	人类提出的重点	智能体执行与反馈	可审计结果
07-10 / 初始化	建立严格 MVVM 项目	创建分层目标、中文说明与部署文档	ddd6458、78f373a
07-11 / 任务列表	完成任务管理和列表投影	实现 ViewModel、搜索、排序及测试	1820f6c、e15050c
07-11 / 依赖	加入前置关系与图投影	实现依赖管理、前置选择和 Graph ViewModel	cb26d91、8103ba3
07-11 / 状态安全	固化合法工作流	修复归档编辑，实现状态机和综合测试	3b45817、6212176
07-12 / 外观与图	完善主题和依赖图	设置持久化、前后继模型、布局修复	bb76760～8308142
07-12 / 生命周期	支持删除与逾期	永久删除、依赖清理和逾期策略	3fc2f1d
07-13 / 批量类别	增加批量操作和分类	原子批量端口、类别领域与测试	81264ad、3a52e8e
07-13 / Widgets	用纯 C++ 前端替换 QML	引入 Contracts、通知、Widgets 壳和主窗口	cbaac08～38c10b1
07-13 / 主流程	迁移任务、类别与依赖	拆分窄 Contract，接通纵向测试	2e36938、90202fd
07-14 / 图迁移	完成 QGraphicsView 依赖图	图元、选择链、缩放和详情联动	72444d1
07-14 / UI 回归	修复布局和输入体验	响应式表单、主题、日期与时长选择器	8954a37～926b7c7
07-14 / 清理	正式移除旧前端	删除 QML 测试、adapter 和 Quick 依赖	05cfdc4
07-14～15 / 维护	提升可维护性	中文注释、投影源和大文件拆分	62e258a～2635460

日期/轮次	人类提出的重点	智能体执行与反馈	可审计结果
07-16 / 统计	实现可解释统计	事件、Service、ViewModel、Charts 页面和修复	c2feb98 ~ 407b012
07-16 / 桌宠	增加第二个 View	动画、窗口、气泡、位置和测试	89ffd48、2e77178
07-17 / 专注底座	实现可靠计时	Schema v5、Repository 与 FocusService	896658e、04805b1
07-17 / 专注页面	接通完整链路	ErrorMapper、ViewModel、页面、导航和测试	2c15754、aad4ed5
07-17 / 桌宠收尾	改善交互与文案	气泡关闭行为和文本修正	63bd12a、2dabe50
07-18 / 验收	提供全面总结和证据	干净构建、39 项测试、Release 部署与报告	本文档

六、团队协作情况

6.1) 分工安排

本项目由两名成员协作完成。在开发过程中，团队采用了“人机协同”的工作模式——人类成员负责架构决策与领域设计，智能体辅助完成编码实现、测试编写。本项目中，刘欣瓚同学主要负责核心代码的产出，石耿同同学主要负责功能性的测试验证

6.2) 人与智能体协同工作

在整个开发周期中，团队成员与智能体形成了紧密的协作关系。具体实践如下：

Info

架构决策由人主导。MVVM 分层边界、状态机转换规则、依赖图的 Finish-to-Start 语义、批量操作的原子事务策略等核心设计均来自团队成员的讨论与判断。智能体不参与架构决策，而是提供“这个设计会导致什么问题”的逆向检验。

Tip

编码实现由智能体高效执行。在架构确定后，具体的 C++ 实现由智能体完成——包括 Service 方法、ViewModel 属性绑定、Qt Widgets 组件布局。人类成员的角色转变为 Code Review：审查生成的代码是否遵守 AGENTS.md 中定义的 MVVM 边界，是否正确处理边界条件。

Warning

测试用例由人和智能体共同设计。人类提出测试场景(如“取消任务不阻塞后继”、“批量归档含不合格任务整批回滚”),智能体编写具体的测试代码(Fake Repository 注入、断言编写)。手工功能测试表由人设计操作步骤,智能体补充边界场景。

七、实际效果

智能体的主要正面效果体现在以下方面:

- 能够在较短周期内完成从领域规则、持久化、ViewModel 到 Widget 的纵向功能闭环。
- 能根据编译错误和测试失败持续迭代,而不是只生成静态代码片段。
- 在大规模前端迁移中维持阶段性目标和回归基线,最终删除旧 QML 依赖。
- 能同步维护架构、迁移和部署文档,使代码决策具备文字依据。
- 通过 Fake、信号测试、内存 SQLite 和架构守卫提高回归发现能力。

八、现存问题

1. 智能体生成的界面需要人工进行视觉验收,自动化测试难以覆盖间距、色彩和高 DPI 下的全部问题。
2. 长任务中可能出现上下文偏移,必须依靠窄任务、阶段性提交和完整测试限制风险。

三、学习回顾与个人总结

一、刘欣瓚

在本次课程中,我承担了项目核心开发者的角色,包括确定项目功能、设计规则、维护 MVVM 边界、审查智能体实现、处理界面反馈。智能体承担了大量检索、样板实现、测试扩充、问题定位和文档整理,但需求取舍、架构规则、交互判断和发布标准仍由我决定。使用智能体没有减少开发者责任,而是把责任从“逐行输入代码”更多转移到“定义问题、建立约束和验证结果”。

通过本次课程,我对 MVVM 的认识从“把代码放进三个目录”深化为“控制依赖方向、状态所有权和变化传播”。Model 拥有状态机、依赖图、事务资格和统计口径;ViewModel 负责可观察投影、输入草稿、命令转发与错误映射;View 只处理控件、绘制、布局、确认和事件转发。Contract 不是新的业务层,而是 ViewModel 向 View 暴露的稳定窄端口。

我也掌握了依赖倒置的实际用法。Service 依赖 Repository 接口而不是 SQLite,Widget 依赖 Contract 而不是具体 ViewModel,只有组合根负责组装具体实现。解耦不是让模块彼此隔绝,而是让合作发生在稳定、强类型、可替换的边界上。状态机与事务让我理解了业务一致性,隐藏按钮不能替代 Service 复核,Service 预检也不能替代数据库事务。只有界面资格、领域规则和持久化原子性共同存在,才能防止过期状态和部分写入。我完成了从 C++ 领域模型到 Qt Widgets 展示的完整实践,能够使用 QObject 信号槽、QAbstractListModel、QStyledItemDelegate、QGraphicsScene、Qt Charts、QSettings、Qt SQL

和 Qt Test，并理解这些 API 应位于哪一层。数据库能力从基础表设计发展到 Schema v5，涵盖任务、依赖、活动事件、专注会话、时间段、唯一活动约束、外键级联、快照和原子状态操作，同时保留旧版本迁移路径。

在项目开发过程中，我也遇到过一些困难。例如，项目最初使用 QML 实现，耦合比较严重，花费了大量的时间精力重构为 Widgets 实现；一些细小的实现智能体不能充分理解我的意图，需要多次对话修改才能满足需求；智能体长对话可能出现上下文偏移，通过维护规则与设计文档，每轮先充分讨论明确开发目标，再使用 Git 小步提交，确保智能体按照我的想法准确执行。

在整个开发过程中，我最大的成长不是掌握更多 Qt API，而是建立系统性工程思维，理解如何规范使用智能体提高开发效率。智能体擅长扩大执行能力，也会放大模糊需求。开发者越能明确目标、边界和证据，产出越可靠；缺少规则与复核时，生成速度只会更快积累技术债。有效的人机协作不是把决定权交给智能体，而是用工程制度让智能体的速度服务于正确方向。

二、石耿同

我在本项目中主要负责功能测试与文档编写。这段经历让我从多个维度获得了显著成长。

在测试方面，我经历了从“编完代码跑一跑”到“系统性测试设计”的转变。项目初期，我只是基于直觉去操作界面，发现哪里不对记录哪里。但随着功能增多和规则复杂化，我意识到必须建立系统化的测试方法：按模块分类、为每条测试设定明确的预期结果、用 git 追溯失败引入的版本。最终形成的 199 项手工测试覆盖了所有功能模块，其中对状态机的边界测试和批量操作的原子性测试尤为关键。

在 MVVM 架构学习方面，我经历了从理论认识到实践理解的跨越。前期只能大致区分 Model、ViewModel、View 三层的基本定义；中期经历了 QML→Widgets 的前端迁移后，我真正理解了为什么 MVVM 的核心不是某个 UI 框架而是依赖方向——View 不应该知道自己绑定的是哪个具体 ViewModel。Contract 抽象层的引入使同一套 ViewModel 可以同时驱动两套 UI 技术栈，这让我对软件架构的“可变性管理”有了切身体会。

在文档编写与团队协作方面，我深刻体会到文档不是“写完代码后的附属品”，而是设计过程的有机组成部分。合理的中文注释帮助智能体更准确地理解业务意图，设计文档在团队讨论和代码评审中起到了“可执行的契约”的作用。

总而言之，最后看到两周之内实现了这样一个项目，还是感到非常的开心的，从中学到了很多。

四、课程改进建议

- 对于前期理论课的讲解过程中，希望根据框架配套更多的例程以及讲解，以及在代码中配套一些对应的注释帮助理解。

- 我认为本课程可以在项目开题时缩小选题范围，最好推荐一些非常适合 MVVM 结构的项目，在开发过程中让同学们尽可能体会到耦合造成的不便，从而理解解耦对于开发的必要性。